1. Input:

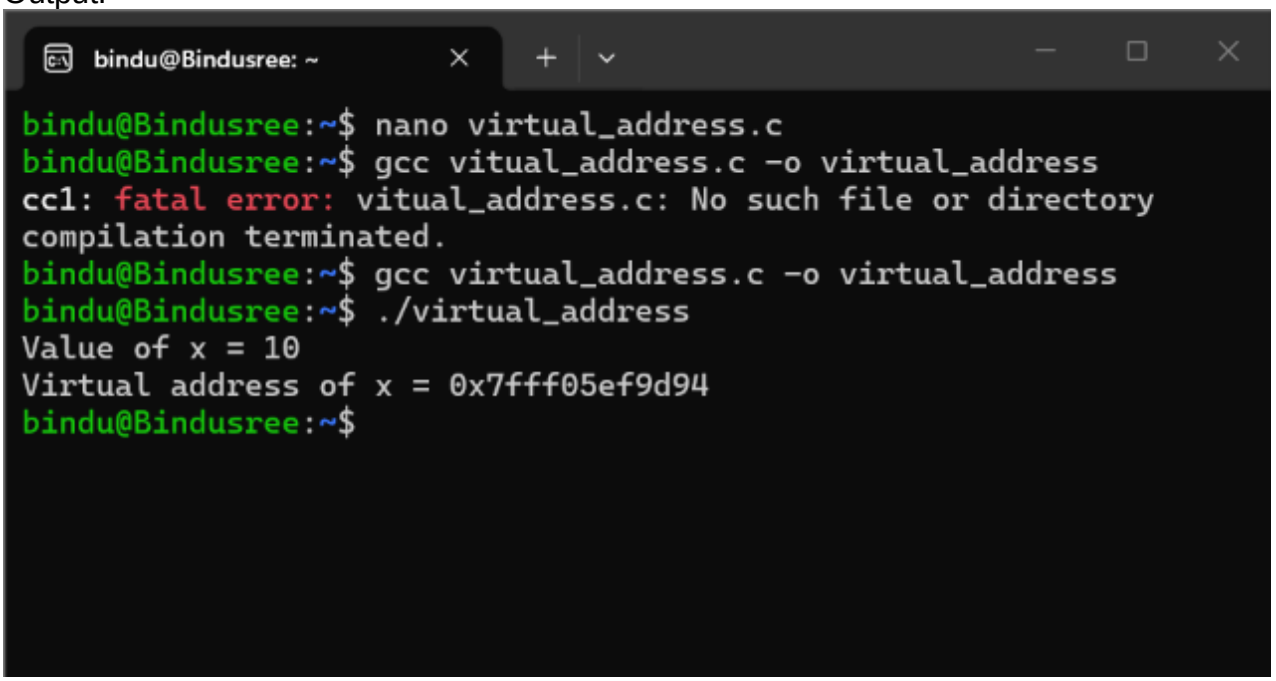
```c
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

Output:



2. Input:

```c
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;
```

Output:

```
bindu@Bindusree:~$ nano memory_segments.c
bindu@Bindusree:~$ gcc memory_segments.c -o memory_segments
bindu@Bindusree:~$ ./memory_segments
Code address  : 0x5b100dff91a9
Data address  : 0x5b100dffc010
BSS address   : 0x5b100dffc018
Heap address  : 0x5b103d650010
Stack address : 0x7ffe259b02bc
bindu@Bindusree:~$
```

3. Input:

```c
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

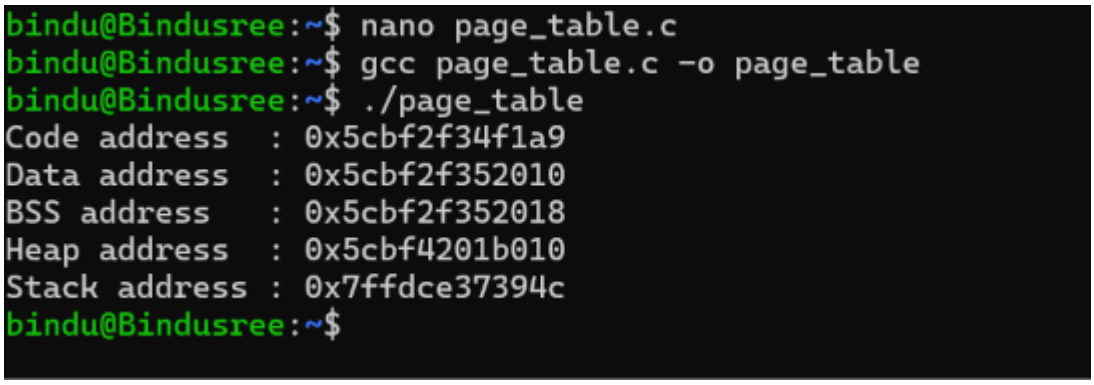
    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

output:


bindu@Bindusree:~$ nano page_table.c
bindu@Bindusree:~$ gcc page_table.c -o page_table
bindu@Bindusree:~$ ./page_table
Code address : 0x5cbf2f34f1a9
Data address : 0x5cbf2f352010
BSS address : 0x5cbf2f352018
Heap address : 0x5cbf4201b010
Stack address : 0x7ffdce37394c
bindu@Bindusree:~$

```c
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address  = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number     = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

output:

```
bindu@Bindusree:~$ nano virtual_to_physical.c
bindu@Bindusree:~$ gcc virtual_to_physical.c -o virtual_to_physi
cal
bindu@Bindusree:~$ ./virtual_to_physical
Virtual Address  = 250
Page Number      = 2
Offset           = 50
Frame Number     = 1
Physical Address = 150
bindu@Bindusree:~$
```

6.Input:

5.Input:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

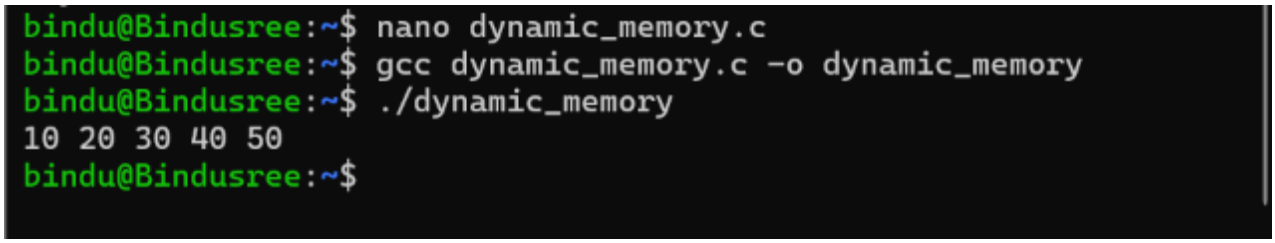
    printf("\n");

    free(p);

    return 0;
}
```

output:

6.Input:


bindu@Bindusree:~$ nano dynamic_memory.c
bindu@Bindusree:~$ gcc dynamic_memory.c -o dynamic_memory
bindu@Bindusree:~$ ./dynamic_memory
10 20 30 40 50
bindu@Bindusree:~$

```c
#include <stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

output:

```
bindu@Bindusree:~$ nano null_pointer.c
bindu@Bindusree:~$ gcc null_pointer.c -o null_pointer
bindu@Bindusree:~$ ./null_pointer
Before accessing memory
Segmentation fault         ./null_pointer
bindu@Bindusree:~$
```

8.Input: